DOCUMENTAZIONE CASO DI STUDIO SMS 2023/24

Gruppo: "TheVerifiers"

A.A. 2023/2024

Professore: **Buono Paolo** paolo.buono@uniba.it

- **Poli Davide** d.poli2@studenti.uniba.it
- **Lauriola Michele** m.lauriola11@studenti.uniba.it
- **Selvaggi Roberto Pio** r.selvaggi3@studenti.uniba.it
- **Papapicco Fabio Vito** f.papapicco7@studenti.uniba.it

SOMMARIO

INTRODUZIONE 3
SCOPO DELL'APPLICAZIONE 3
PANORAMICA DELLE FUNZIONALITA' PRINCIPALI 3
REQUISITI DI SISTEMA 4
DESIGN DELL'INTERFACCIA UTENTE 5
IMPLEMENTAZIONE 6
STRUMENTI UTILIZZATI PER LO SVILUPPO 6
PRINCIPALI COMPONENTI DELL'APP 7
VISUALIZZA PROFILO PAZIENTE 7
VISUALIZZA PROFILO DOTTORE 8
VISUALIZZAZIONE COSTI 8
AGGIUNGI COSTI 8
PANIC BUTTON 9
INFORMAZIONI CENTRI DI ACCOGLIENZA 9
POSTI DI INTERESSE 9
INFORMAZIONI SULLA CITTA' 10
LISTA UTENTI 10
AGGIUNGI UTENTI 11
QR_CODE 12
VIDEO 12
STORICO MISURAZIONI 13
BLUETOOTH 13
SENSORE 15
DOCUMENTI 15
DESCRIZIONE DATABASE 17
USER 17
COSTS 17
MISURAZIONI 17
CENTRI DI ACCOGLIENZA 18
STORAGE 18
PROSPETTIVE FUTURE 19

INTRODUZIONE

SCOPO DELL'APPLICAZIONE

L'AsilApp è concepita con l'obiettivo primario di fornire una soluzione completa per la gestione dei richiedenti asilo, ponendo particolare attenzione alla semplificazione e al miglioramento dell'interazione tra gli utenti e i loro dottori, oltre a promuovere attivamente la salute degli stessi. Questa piattaforma innovativa offre una serie di funzionalità essenziali, mirate a ottimizzare l'esperienza degli utenti e a facilitare l'accesso ai servizi di assistenza sanitaria e sociale.

PANORAMICA DELLE FUNZIONALITA' PRINCIPALI

In primo luogo, l'app fornisce una sezione dedicata al riconoscimento dell'utente, che può essere realizzato anche tramite QR code, garantendo un'identificazione sicura e rapida. Successivamente, vengono raccolti e gestiti i dati anagrafici dell'utente attraverso una scheda personale, consentendo un archivio completo e aggiornato delle informazioni personali.

Una caratteristica distintiva dell'AsilApp è la sezione dedicata ai parametri medici, che consente la rilevazione e la registrazione dei dati biometrici utilizzando apparecchiature biomedicali presenti in un contenitore dedicato. Questi dati vengono visualizzati in forma di singole registrazioni, consentendo agli utenti e ai professionisti sanitari di monitorare i trend e la salute generale dell'utente nel tempo.
Per quanto riguarda la sezione informativa, l'app offre una vasta gamma di contenuti utili, tra cui video personalizzati, informazioni sui centri di accoglienza e i servizi offerti, nonché una panoramica sulla città o regione ospitante e i principali servizi disponibili nelle vicinanze.

L'app offre strumenti per la gestione delle spese associate a ciascun utente, consentendo la registrazione e il monitoraggio dei costi relativi a prodotti e servizi, con la possibilità di eseguire semplici interrogazioni per tenere traccia delle spese nel tempo.

Infine, l'AsilApp offre un approccio integrato alla gestione della salute e delle necessità quotidiane.

REQUISITI DI SISTEMA

I requisiti di sistema dell’app sono i seguenti:

Sistema operativo: L'app richiede un dispositivo Android con una versione di sistema operativo Android 8.0 (Oreo) o successivo.

Memoria RAM: L'app richiede una quantità minima di memoria RAM di 2 gb per gestire efficacemente le operazioni dell'app e evitare ritardi o blocchi.

Processore: L'app richiede almeno un processore quad-core con una frequenza di clock di almeno 1,5 GHz per garantire una buona velocità e prestazioni dell'app.

Spazio di archiviazione: L'app richiede uno spazio di archiviazione interno adeguato ad essere installata e salvare i dati associati. Un minimo di 100 MB di spazio libero è consigliato, ma potrebbe essere necessario più spazio a seconda delle dimensioni e delle esigenze dell'app.

Connettività: L'app richiede funzionalità di connettività essenziali, come il supporto per Wi-Fi o la connettività dati, per agevolare il caricamento, il download e la condivisione dei dati dell'app, nonché per consentire l'accesso a servizi esterni, se necessario. È fondamentale anche l'integrazione di un modulo GPS, che permette la condivisione della posizione o l'attivazione di altre funzionalità basate sulla localizzazione. Infine, il dispositivo deve essere dotato di un modulo Bluetooth integrato per facilitare la connessione e la comunicazione con dispositivi compatibili Bluetooth.

Fotocamera: Una fotocamera integrata per scansionare il QR code e riconoscere un utente specifico.

Permessi: L’app richiede anche dei permessi, come l’accesso alla fotocamera, alla posizione, accesso a internet e bluetooth.

DESIGN DELL'INTERFACCIA UTENTE

ASPETTO VISIVO DELL'APP

L'interfaccia di AsilApp è stata progettata per offrire una navigazione intuitiva e una piacevole user experience, si è deciso di disporre gli elementi in modo logico e intuitivo, senza appesantire troppo le singole interfacce e garantendo alta usabilità.

I pulsanti e le icone sono stati progettati con uno stile accattivante e facilmente riconoscibile: bordi principalmente arrotondati, per un senso di leggerezza, e utilizzando i colori distintivi del nostro gruppo.

Palette dei colori utilizzati:

Tra i colori principalmente utilizzati abbiamo il verde (#068251). Oltre ad essere il nostro "marchio di fabbrica", è il colore della natura, spesso associato alla salute e alla crescita. L'esposizione al verde può avere un effetto calmante e ridurre lo stress. Inoltre, è associato alla fertilità, quindi può avere un impatto positivo sul benessere generale.

Insieme al verde, la squadra ha deciso di creare un contrasto visivo morbido ed equilibrato. Per far fede a questa scelta, il secondo colore utilizzato è il bianco (#FFFFFF).
Se il verde porta una sensazione di natura e calma, allora il bianco aggiunge luminosità e pulizia. Questo contrasto può rendere l'ambiente piacevole e rilassante per gli occhi.

Per i testi si è deciso di utilizzare il classico nero (#000000). Questa scelta è stata intenzionale al fine di conferire un contrasto visivo e migliorare l'usabilità dell'app.

Le transizioni e le animazioni sono state utilizzate per migliorare l'esperienza utente senza però distogliere l'attenzione dagli elementi principali. Principalmente abbiamo utilizzato le slides per dare un senso di movimento in entrata e uscita, così da rendere l'utente consapevole di ciò che è successo.

IMPLEMENTAZIONE

STRUMENTI UTILIZZATI PER LO SVILUPPO

- **DESCRIZIONE DELLE TECNOLOGIE E DEGLI STRUMENTI**

Questa sezione fornisce una descrizione completa delle tecnologie e degli strumenti impiegati durante lo sviluppo dell'app Android “AsilApp”. Queste informazioni sono cruciali per comprendere l'architettura, le funzionalità e le scelte tecniche adottate nel processo di sviluppo dell'app.

- **LINGUAGGIO DI PROGRAMMAZIONE:**

L'app è stata sviluppata utilizzando il linguaggio di programmazione Java ovvero una delle principali scelte per lo sviluppo di app Android visto che offre una sintassi intuitiva ed una vasta libreria che facilita la creazione di app affidabili e performanti.

- **AMBIENTE DI SVILUPPO INTEGRATO (IDE):**

Lo sviluppo dell'app è stato effettuato utilizzando Android Studio, un potente ambiente di sviluppo integrato (IDE) specificamente progettato per lo sviluppo di app Android.

- **VERSIONE DELLE COMPONENTI ANDROID:**

L'app è stata progettata per supportare una vasta gamma di dispositivi Android. La versione minima di Android (minSdk) supportata dall'app è la 26. La versione di compilazione (compileSdk) utilizzata è la 34, che offre accesso alle ultime funzionalità e ottimizzazioni del sistema operativo Android. L'app è stata destinata alla versione 32 (targetSdk) di Android, garantendo maggior compatibilità.

- **STRUMENTO DI BUILD DEL PROGETTO:**

Per la gestione delle dipendenze e la compilazione del progetto, è stato utilizzato Gradle nella versione 8.2.2.

- **DATABASE:**

L'app utilizza Firebase ovvero una piattaforma di sviluppo di app mobile basata sul cloud offerta da Google. Firebase fornisce un potente database in grado di gestire e sincronizzare i dati in modo efficiente tra gli utenti dell'app.

- **VERSION CONTROL SYSTEM:**

Per il controllo di versione del codice sorgente, è stato utilizzato Git (che consente di tenere traccia delle modifiche al codice e facilita la collaborazione tra i membri del gruppo).

- **STRUMENTO DI SIMULAZIONE:**

Durante il processo di sviluppo e test, è stato utilizzato l'Android Virtual Device (AVD) e due dispositivi fisici per simulare l'esecuzione dell'app su un dispositivo Android. In particolare, è stato configurato un dispositivo Pixel XL con API 34, consentendo di testare l'app in un ambiente simulato che rispecchia le caratteristiche di tale dispositivo e versione di Android.

PRINCIPALI COMPONENTI DELL'APP

VISUALIZZA PROFILO PAZIENTE

L'activity UserProfileActivity permette di visualizzare il profilo (paziente). Durante la sua creazione, viene vengono inizializzati gli elementi dell'interfaccia utente, come l'icona del QR code e il pulsante di modifica del profilo. Durante l'avvio dell'attività, viene istanziato un oggetto UserProfileAdapter per gestire la visualizzazione del profilo utente paziente. Viene inoltre impostato un listener sul click dell'icona del QR code che apre il fragment MyQrCodeFragment, passando l'ID dell'utente corrente. Quando l'utente preme il pulsante di modifica del profilo, viene avviata l'attività ModifyProfileActivity, dove sarà possibile modificare solo alcuni campi del suo profilo.

La classe UserProfileAdapter è responsabile per la gestione dell'adapter per la visualizzazione del profilo utente paziente. Inizializza vari TextView per visualizzare le informazioni del profilo utente, come nome, cognome, data di nascita, ecc. Il metodo setProfileView(Paziente paziente) viene utilizzato per popolare i TextView con le informazioni del paziente passato come parametro. Inoltre, il metodo findReceptionCenter cerca il centro di accoglienza associato al paziente e lo imposta nel TextView corrispondente, utilizzando il database Firestore.

L'activity ModifyProfileActivity gestisce l'attività per la modifica del profilo dell'utente paziente. All'avvio, vengono inizializzati vari elementi dell'interfaccia utente, come campi EditText per inserire il nome, il cognome, il numero di telefono, ecc. Inoltre, vengono configurati spinner e autocompletamento per la selezione del genere e della nazionalità. Viene inoltre recuperato l'elenco dei centri di accoglienza da Firebase e popolato uno spinner con i nomi dei centri. Quando l'utente preme il pulsante di salvataggio, vengono validati i dati inseriti e, se corretti, vengono aggiornati nel database Firestore.

VISUALIZZA PROFILO DOTTORE

L’activity DocProfileActivity permette di mostrare il profilo (dottore). Durante la sua creazione, vengono inizializzati vari elementi dell'interfaccia utente, come TextView per visualizzare il nome, il cognome, il genere, ecc. Viene inoltre inizializzato il pulsante per la modifica del profilo. Successivamente, vengono recuperati i dati del profilo passati tramite un Intent e vengono impostati nei rispettivi TextView. Infine, viene impostato un listener sul pulsante di modifica del profilo, che reindirizza l'utente all'attività `ModifyProfileDoc` per apportare eventuali modifiche al profilo.

L’activity ModifyProfileDoc gestisce l'attività per la modifica del profilo del dottore. All'avvio, vengono inizializzati vari elementi dell'interfaccia utente, come campi EditText per inserire il nome, il cognome, l'indirizzo dello studio, ecc. Viene inoltre recuperato l'oggetto PersonaleAutorizzato passato dall'activity precedente. Quando l'utente preme il pulsante di salvataggio, vengono validati i dati inseriti e, se corretti, vengono aggiornati nel database Firestore.

VISUALIZZAZIONE COSTI

L'activity " CostsActivity " consente di visualizzare, filtrare e gestire tutti i costi dell'utente corrente, fornendo funzionalità come la ricerca, il filtraggio per data e categoria.

Il metodo eventChangeListener() crea un listener per i cambiamenti nel documento "costs" del database. Quando vengono rilevati cambiamenti, vengono aggiunti gli oggetti "Costs" corrispondenti all'ArrayList allCostsArrayList. Successivamente, vengono notificati all'adattatore i cambiamenti per aggiornare la RecyclerView con i nuovi dati.

Infine, i metodi filterListByDate() e filterListByCategory() vengono utilizzati per filtrare l'elenco dei costi in base a diversi criteri come la data e la categoria. Questi metodi creano nuove liste di costi filtrate in base ai criteri specificati e viene aggiornano l'adattatore della RecyclerView per mostrare solo gli elementi filtrati.

AGGIUNGI COSTI

L’activity AddCostActivity consente di creare un nuovo costo relativo a un determinato utente, potendo specificare anche la categoria del costo sostenuto.

È stato introdotto un controllo per I campi obbligatori, per evitare di far inserire all’utente informazioni incomplete relative alle spese.
Infine, viene creato un map per immagazzinare le informazioni prese, tramite l’intent, nel database. Quindi, il map viene usato per permettere il salvataggio di quelle informazioni.

PANIC BUTTON

L'activity "PanicButtonActivity" fornisce agli utenti un pulsante di emergenza per richiedere assistenza rapida in situazioni di emergenza. Premendo il pulsante di emergenza, l'utente può effettuare una chiamata diretta al numero preimpostato per ricevere assistenza immediata.

All'avvio dell'activity, vengono inizializzati il pulsante di emergenza e il pulsante per accedere ai numeri utili. Viene impostato un listener sul pulsante di emergenza per gestire l'azione di chiamata in caso di emergenza. Inoltre, è gestito un listener sul pulsante per accedere ai numeri utili, che apre un fragment contenente una lista di numeri utili.

È implementata la gestione dei permessi per effettuare chiamate telefoniche. Se l'utente ha concesso il permesso, la chiamata viene avviata direttamente. In caso contrario, viene richiesto all'utente di concedere il permesso.

INFORMAZIONI CENTRI DI ACCOGLIENZA

L’activity InfoCentroActivity, fornisce agli utenti informazioni dettagliate sul centro di accoglienza, inclusi dettagli sulle regole, i servizi, l'ubicazione e i contatti, attraverso diverse schede gestite da un ViewPager2 e un TabLayout.
Con il fragment ServiceRuleFragment è possibile mostrare le regole e i servizi offerti dal centro di accoglienza. Recupera i dati da due file XML ("rules.xml" e "services.xml") attraverso un'istanza della classe DBCentroAccoglienza, li elabora utilizzando la classe XMLUtility e li visualizza tramite un RecyclerView.

Con il fragment WhereFragment è possibile mostrare l'ubicazione del centro di accoglienza. Recupera l'ubicazione dal database utilizzando la classe DBCentroAccoglienza e visualizza le informazioni relative alla via, città, provincia e regione.

Infine, con il fragment WhoFragment possiamo mostrare le informazioni sul centro di accoglienza come nome, email, telefono e immagini. Recupera i dati dal database utilizzando la classe DBCentroAccoglienza e visualizza le informazioni sul nome e i contatti. Inoltre, mostra un'immagine slider con le immagini del centro.

POSTI DI INTERESSE

L'activity "PlacesOfInterestActivity" consente agli utenti di visualizzare una mappa interattiva con i luoghi di interesse nelle vicinanze. Utilizzando la posizione corrente dell'utente, la mappa mostra i luoghi come ospedali, scuole e ristoranti nelle vicinanze.

All'avvio dell'activity, viene inizializzata la mappa e vengono configurati i pulsanti per visualizzare i diversi tipi di luoghi di interesse. È implementata la funzionalità per ottenere la posizione corrente dell'utente e aggiornare la mappa di conseguenza.

La posizione corrente dell'utente viene determinata utilizzando il servizio di localizzazione del dispositivo Android. Il marker sulla mappa indica la posizione attuale dell'utente.

Inoltre, è gestito un listener sui marcatori sulla mappa per fornire indicazioni stradali verso un luogo selezionato. L'utente può fare clic su un marker che rappresenta un luogo di interesse per visualizzare le indicazioni stradali dalla posizione corrente dell'utente al luogo selezionato. Le indicazioni sono calcolate utilizzando le API di Google Directions.

Sono presenti pulsanti per ingrandire e ridurre la mappa, permettendo all'utente di regolare il livello di zoom secondo le proprie preferenze.
Se l'applicazione non dispone dei permessi di localizzazione, verrà richiesto all'utente di concederli per poter utilizzare tutte le funzionalità legate alla localizzazione.

INFORMAZIONI SULLA CITTA'

L'activity "WikiCityInfoActivity" consente agli utenti di accedere a informazioni sulla città selezionata da Wikipedia. Utilizzando la posizione corrente dell'utente, l'activity esegue una ricerca geografica su Wikipedia per trovare informazioni sulla città e visualizzarle tramite un WebView.

All'avvio dell'activity, viene abilitata la posizione e avviata la ricerca delle informazioni sulla città su Wikipedia. Utilizzando la libreria OkHttpClient, l'activity esegue una richiesta HTTP per ottenere i dati di Wikipedia sulla città. Una volta ottenuti i dati, vengono visualizzati all'interno di un WebView per consentire agli utenti di leggere le informazioni sulla città direttamente dall'applicazione.

LISTA UTENTI

Questa activity consente ai dottori di visualizzare una lista dei loro pazienti. Una lista di pazienti prelevati dal database Firestore che offre anche la possibilità di essere filtrata utilizzando una searchview e infine consente anche l'aggiunta di un paziente cliccando sul pulsante affianco alla searchview.

Per l'implementazione della searchview è stato impostato il listener SearchView.OnQueryTextListener. Questo listener gestisce gli eventi di ricerca. La funzione

onQueryTextSubmit() viene chiamata quando l'utente invia la query di ricerca, mentre la funzione onQueryTextChange() viene chiamata ogni volta che il testo di ricerca cambia. Inoltre, in questa funzione, viene chiamata filterList() per filtrare la lista degli utenti in base al testo di ricerca.

La funzione eventChangeListener(), invece, crea un listener per ascoltare gli eventi sulla collection "User " nel database Firestore. I documenti vengono ordinati per campo "name" in ordine ascendente utilizzando orderBy("name", Query.Direction.ASCENDING). Dopo di che viene applicato un filtro per recuperare solo gli animali che hanno un campo "emailDoc" (id proprio dottore) uguale all'email del dottore corrente. Viene aggiunto un listener addSnapshotListener() per gestire gli eventi sui risultati della query; quindi, quando viene aggiunto un nuovo documento alla collezione, viene creato un oggetto Paziente utilizzando dc.getDocument().toObject(Paziente.class) e viene aggiunto userArrayList. Infine, viene chiamato listUserAdapter.notifyDataSetChanged() per aggiornare l'Adapter del RecyclerView con i nuovi dati.

Per quanto riguarda l’dapter invece viene utilizzato per visualizzare una lista di utenti. All'interno dell’adapter, vengono implementati i metodi richiesti per un RecyclerView.Adapter, come onCreateViewHolder() e onBindViewHolder(). Questi metodi gestiscono la creazione delle visualizzazioni degli elementi della lista e il collegamento dei dati a ciascuna visualizzazione.

AGGIUNGI UTENTI

Se si clicca sul ImageView addUser si viene mandati all’activity AddUserActivity dove ogni dottore ha la possibilità di inserire un nuovo paziente.
In questa activity si raccolgono i dati inseriti dal dottore e vengono immagazzinati in una Map che poi verranno mandati al database per effettuare la registrazione del nuovo paziente.
Viene impostata una password di default che poi attraverso il metodo auth.sendPasswordResetEmail(emailDB) verrà mandata una mail al paziente con la possibilità di impostare la nuova password.

Il metodo showDatePickerDialog(), si occupa di mostrare un DatePickerDialog per consentire all'utente di selezionare una data di nascita. Ottiene la data corrente utilizzando un oggetto Calendar, estrae l'anno, il mese e il giorno dalla data corrente.

Istanzia un oggetto DatePickerDialog, impostando come contesto l'activity corrente e passando un DatePickerDialog.OnDateSetListener per gestire l'evento di selezione della data, nel metodo onDateSet dell'OnDateSetListener, viene creato un oggetto Calendar per la data selezionata dall'utente. Controlla se la data selezionata è successiva alla data corrente.

Con il metodo datePickerDialog.getDatePicker().setMaxDate(System.currentTimeMillis()) imposta il limite massimo della data selezionabile nel DatePickerDialog alla data corrente, in modo che l'utente non possa selezionare una data futura.

La funzione showDatePickerDialog permette di salvare la data corrente ed imposta un limite per il giorno di nascita in modo tale da non superare il giorno corrente.
La funzione loadCentriAccoglienza(), si occupa di caricare i dati dei centri di accoglienza da un database Firestore e popolare uno Spinner con i nomi dei centri di accoglienza recuperati.
Istanzia un oggetto DBCentroAccoglienza per interagire con il database dei centri di accoglienza.

Ottiene l'istanza della collezione dei centri di accoglienza tramite il metodo getCollectionInstance() di DBCentroAccoglienza, Esegue una query per recuperare tutti i documenti nella collezione dei centri di accoglienza e popola una lista di CentroAccoglienza con gli oggetti recuperati.
Crea un ArrayAdapter utilizzando la lista di centri di accoglienza appena popolata ed imposta l'adapter appena creato allo Spinner centroAccoglienza.

QR_CODE

L'activity QrCodeReaderActivity gestisce l'attività per la scansione dei codici QR. All'avvio, inizializza il pulsante per avviare la scansione. Quando l'utente preme il pulsante, viene controllato se è stata concessa l'autorizzazione per l'uso della fotocamera. Se sì, viene avviata la scansione del codice QR; altrimenti, viene richiesto il permesso. Una volta scansionato il codice QR, viene visualizzata una scheda popup contenente le informazioni dell'utente corrispondente al codice QR scansionato.

La classe UserCardAdapter fornisce metodi per ottenere le informazioni dell'utente corrispondente a un determinato ID o email dal database e popolare una scheda utente (UserCard) con tali informazioni. Viene utilizzata per ottenere i dettagli dell'utente dopo la scansione del codice QR. Utilizza il database DBUtenti per recuperare le informazioni dell'utente, quindi popola la scheda utente e la mostra come popup usando la classe UserCard.

Questa classe rappresenta la scheda utente che viene visualizzata come popup dopo la scansione del codice QR. Contiene campi per il nome, il cognome dell'utente e un pulsante per navigare al profilo completo dell'utente. Quando viene visualizzata la scheda, la visualizzazione del codice QR sottostante viene nascosta. Ha anche un pulsante per chiudere la scheda.

VIDEO

L'activity VideoActivity si occupa della gestione dei video utili per i pazienti. Entrando nell'apposita sezione "Video", l'utente sarà in grado di visualizzare alcuni filmati proposti affinchè

possa saperne di più sul paese ospitante e sulla sua condizione. Per ottenere i filmati si utilizza un ArrayList nel quale vengono caricati gli URL dei video consigliati, con i relativi titoli. L'array contiene degli oggetti appartenenti alla classe YouTubeVideos, fondamentale perla visualizzazione del video e del titolo.

VideoAdapter

è progettata per gestire la visualizzazione dei video di YouTube all'interno di una RecyclerView in un'applicazione Android. Quando viene creata un'istanza di questo adattatore e associata alla RecyclerView, si occupa di inflare il layout per ciascun elemento, associare i dati a ciascun ViewHolder e gestire i clic sull'icona del play (utilizzando un'interfaccia VideoClickListener.).

"onBindViewHolder" imposta il titolo del video nella TextView videoTitle del ViewHolder.

"onCreateViewHolder" crea una nuova istanza di VideoViewHolder quando necessario. Utilizza un LayoutInflater per creare una vista inflando il layout video.xml.

STORICO MISURAZIONI

L'activity " StoricoMisurazioniActivity" consente di visualizzare, filtrare e gestire tutte le misurazioni effettuate dal dottore sull'utente corrente, fornendo funzionalità come il filtraggio per data e categoria.

Il metodo eventChangeListener() crea un listener per i cambiamenti nel documento "misurazioni" del database. Quando vengono rilevati cambiamenti, vengono aggiunti gli oggetti "Misurazioni" corrispondenti all'ArrayList allMeasurementArrayList.

Successivamente, vengono notificati all'adattatore i cambiamenti per aggiornare la RecyclerView con i nuovi dati.

Infine, i metodi filterListByDate() e filterListByCategory() vengono utilizzati per filtrare l'elenco delle misurazioni in base a diversi criteri come la data e la categoria.

Questi metodi creano nuove liste di misurazioni filtrate in base ai criteri specificati e viene aggiornano l'adattatore della RecyclerView per mostrare solo gli elementi filtrati.

BLUETOOTH

La classe `BluetoothActivity` costituisce un'interfaccia utente per gestire le funzionalità Bluetooth del dispositivo, incluso l'avvio della scansione, la ricerca dei dispositivi Bluetooth nelle vicinanze e l'accoppiamento con essi. Inoltre, abilita anche la localizzazione, necessaria per garantire il corretto funzionamento del Bluetooth su alcune versioni di Android.

Attraverso il metodo `requestBluetoothPermission`, l'utente viene sollecitato ad attivare il Bluetooth e la localizzazione, se non sono già attivi, utilizzando i metodi `enableBluetooth()` ed `enableLocation()`. Questo passaggio è fondamentale per permettere all'applicazione di eseguire la scansione dei dispositivi Bluetooth nelle vicinanze.

Vengono configurati dei listener per gestire il pulsante di ricerca e gli elementi della lista dei dispositivi rilevati. Quando l'utente fa clic sul pulsante di ricerca, viene richiamato il metodo `searchForDevices()`, che avvia la ricerca dei dispositivi Bluetooth disponibili e aggiorna dinamicamente la lista dei dispositivi rilevati `arrayListBluetoothDevices` e l'adapter associato, consentendo all'utente di visualizzare i dispositivi in tempo reale sulla ListView.

Una volta visualizzata la lista dei dispositivi rilevati, l'utente può selezionare un dispositivo facendo clic su di esso. In tal caso, viene chiamato il metodo `handleDetectedItemClick()`. Questo metodo gestisce il clic sugli elementi rilevati nella lista dei dispositivi Bluetooth. Se il dispositivo è già accoppiato, il metodo mostra un messaggio di avviso, disaccoppia il dispositivo e avvia nuovamente il processo di accoppiamento. Se il dispositivo non è già accoppiato, mostra un messaggio di avviso e avvia direttamente il processo di accoppiamento.

Successivamente, il metodo connectToDeviceWithPairing() viene utilizzato per gestire la connessione al dispositivo Bluetooth selezionato. Questo metodo avvia un nuovo thread per eseguire l'operazione in modo asincrono, accoppiando prima il dispositivo con il PIN fornito utilizzando il metodo pairDevice(). Una volta completato l'accoppiamento, il metodo crea un socket Bluetooth per la comunicazione con il dispositivo utilizzando il metodo createBluetoothSocket().

Il metodo createBluetoothSocket() gestisce l'accoppiamento del dispositivo attraverso il metodo pairDevice() e crea il socket Bluetooth per la comunicazione. Inoltre, gestisce anche la connessione al dispositivo selezionato, stabilendo una connessione tramite il socket Bluetooth creato.

Una volta completato l'accoppiamento, viene mostrato un popup attraverso il metodo `showCustomPopup()`, consentendo all'utente di effettuare le misurazioni e di avviare il sensore facendo clic sul pulsante "Tremore".

Infine, una volta terminate le misurazioni, il dispositivo si disassocia automaticamente utilizzando il metodo `dissociateDevice()`, consentendo all'utente di eseguire nuovamente l'operazione in seguito.

SENSORE

La classe SensorHandler gestisce il sensore giroscopico del dispositivo Android per rilevare eventuali tremori. Questa classe implementa l'interfaccia SensorEventListener per ascoltare gli eventi del sensore.

All'interno della classe, viene inizializzato il SensorManager e ottenuto il riferimento al sensore giroscopico. Viene fornito un costruttore che accetta un contesto e un callback per gestire i risultati ottenuti dal sensore.

Il metodo startSensor() viene utilizzato per avviare il sensore. Se il sensore non è già attivo e il sensore giroscopico è disponibile, viene registrato un listener per rilevare i cambiamenti nel sensore. Dopo un intervallo di tempo predefinito (10 secondi), il sensore viene disattivato e vengono valutati i dati raccolti.

All'interno del metodo onSensorChanged(), vengono acquisiti i valori di accelerazione lungo gli assi x, y e z e memorizzati in una lista di dati del sensore.

Il metodo evaluateSensorData() viene utilizzato per valutare i dati del sensore raccolti. Se non sono stati rilevati dati, viene chiamato il callback con un messaggio appropriato. Altrimenti, viene calcolato il valore RMS (Root Mean Square) dei dati del sensore e confrontato con una soglia modificata. Se il valore RMS supera la soglia, viene segnalato un tremore tramite il callback, altrimenti viene segnalato l'assenza di tremori.

Infine, la classe fornisce un'interfaccia SensorCallback che definisce il metodo onResult() per gestire i risultati ottenuti dal sensore, fornendo un feedback all'utente sull'attività rilevata.

DOCUMENTI

L’activity DocumentDocActivity permette di visualizzare tutti i documenti associati a un paziente. Utilizza un adapter (DocumentAdapter) per popolare un RecyclerView con i documenti. Inoltre, gestisce la ricerca dei documenti tramite una SearchView. Quando l'utente preme il pulsante per aggiungere un documento, viene avviata l’activity (AddDocumentDocActivity) per selezionare e caricare un nuovo documento.

In questa activity viene gestita l'aggiunta di un nuovo documento associato a un paziente. L'utente può selezionare un file PDF dalla memoria del dispositivo e caricarlo nel Firebase Storage. Inoltre, fornisce funzionalità per visualizzare, condividere o eliminare i documenti già caricati.

L’adapter DocumentAdapter gestisce la visualizzazione dei documenti all'interno di un RecyclerView. Ogni elemento del RecyclerView corrisponde a un documento, e l'adapter gestisce le azioni associate ai pulsanti (download, visualizzazione, condivisione, eliminazione) per ciascun documento. Inoltre, fornisce un metodo per filtrare la lista dei documenti in base a una query di ricerca.

Gestione del click sul pulsante "Download", quando si fa clic su questo pulsante, viene verificato se l'applicazione ha il permesso di scrivere nello storage esterno del dispositivo.
Se il permesso non è stato concesso, viene richiesto all'utente di concedere il permesso, una volta ottenuto il permesso, viene creato un riferimento allo storage Firebase che punta al documento desiderato. Viene quindi specificato il percorso locale in cui salvare il file scaricato (localFile), che di solito è la cartella "Download" sullo storage esterno. Infine, il file viene scaricato da Firebase Storage nel percorso locale specificato e viene mostrato un messaggio di successo o di fallimento tramite Toast.

Gestione del click sul pulsante "View”, quando si fa clic su questo pulsante, viene creato un riferimento allo storage Firebase che punta al documento desiderato, viene quindi specificato il percorso locale in cui salvare temporaneamente il file (localFile), che di solito è la directory privata dell'applicazione. Successivamente, il file viene aperto utilizzando un'azione di visualizzazione predefinita del sistema, che consente all'utente di visualizzare il documento. Se non è possibile aprire il documento, viene mostrato un messaggio di errore tramite Toast.

Gestione del click sul pulsante "Share", quando si fa clic su questo pulsante, viene creato un riferimento allo storage Firebase che punta al documento desiderato, viene quindi specificato il percorso locale del file (localFile) e viene creato un'intenzione di condivisione per condividere il file tramite altre app sul dispositivo. L'intenzione di condivisione include il tipo di dati del file (application/pdf) e il percorso del file stesso. viene quindi avviata un'attività di scelta per consentire all'utente di selezionare l'applicazione con cui desidera condividere il documento.
Gestione del click sul pulsante "Delete", quando si fa clic su questo pulsante, viene creato un riferimento allo storage Firebase che punta al documento da eliminare. Viene quindi chiamato il metodo delete () sul riferimento per eliminare il file dallo storage Firebase. Se l'eliminazione ha successo, il documento viene rimosso dalla lista visualizzata nell'adapter e viene mostrato un messaggio di conferma tramite Toast. In caso di fallimento dell'eliminazione, viene mostrato un messaggio di errore tramite Toast.

DESCRIZIONE DATABASE

Firebase è una piattaforma serverless, ideale per lo sviluppo di applicazioni mobili e web, che sfrutta un modello di database NoSQL. A differenza dei tradizionali sistemi di gestione di database relazionali, il NoSQL non segue lo schema rigido del linguaggio SQL per interrogare i dati. Invece, Firebase organizza i dati in collezioni e documenti, creando una struttura flessibile e scalabile. Le principali entità utilizzate in Firebase sono le collezioni, che fungono da contenitori per i documenti, ciascuno dei quali può contenere campi e valori associati.

Le raccolte utilizzate sono:

- User;
- Centri di accoglienza;
- Costs;
- Misurazioni;

Ogni raccolta contiene al suo interno documenti, i quali sono essenzialmente composti da campi ai quali vengono associati valori.

USER

La raccolta user contiene tutti i documenti relativi agli utenti registrati, indipendentemente dal tipo.

Sono quindi presenti i documenti relativi a “user” e “personale autorizzato”.

I campi presenti sono quindi personalizzati in base al tipo di utente.

USER: String id, String name, String surname, String gender, String birthdate, String placeBirth, String nationality, String receptioncenter, String email, String telefono, String emailDoc, String idDoc, String type, String password.

DOC: String id, String name_center, String name, String gender, String surname, String e-mail, String password, String address, String phone.

COSTS

La raccolta costs contiene tutti i costi registrati da un utente e questi costi riguardano tutte quelle che sono le spese associate alla cura di un utente.

Ogni documento presenta i seguenti campi: String id, String name, String category, String quantity, String price, String userid, String date, String userEmail.

MISURAZIONI

La raccolta user contiene tutti i documenti relativi alle misurazioni effettuate sugli utenti registrati.

I campi presenti sono quindi: String id, String data, String categoria, String risultato, String userid, String userEmail.

CENTRI DI ACCOGLIENZA

La raccolta user contiene tutti i documenti relativi ai centri di accoglienza.

I campi presenti sono quindi: String nameId, String nome, String infoutils, Map<String, String> contatti, Map<String, String> località, List<String> imgReference, List<String>rulesServices.

STORAGE

Firebase offre anche un servizio di cloud storage, che consente di archiviare file di grandi dimensioni come immagini, video e documenti.

Lo storage è stato suddiviso in:

- documenti: qui sono presenti tutti i documenti inseriti dal dottore raccolti in cartelle avendo il path: “documenti- id dell’utente”.
- centri_di_accoglienza: contiene le immagini dei vari centri di accoglienza.

PROSPETTIVE FUTURE

Oltre alle funzionalità esistenti descritte in precedenza, AsilApp ha ambiziosi piani per il futuro, con l'introduzione di nuove e innovative caratteristiche che miglioreranno l'esperienza degli utenti e sosterranno i richiedenti asilo e gli utenti del nostro prodotto. Le funzionalità future dell'app mirano a migliorare
l'esperienza utente, offrendo strumenti avanzati e interazioni sociali
che permettono agli utenti di connettersi e ricevere
supporto costante, magari aggiungendo un chat-bot integrato.

- **Sistema di Allerta e Notifiche Personalizzate:**

Implementazione di un sistema di allerta per i dottori, avvisando in tempo reale su situazioni mediche o cambiamenti significativi nei parametri vitali dei pazienti.
Notifiche personalizzate per i pazienti, fornendo promemoria per visite mediche, prescrizioni di farmaci o follow-up.

- **Messaggistica In-App per Supporto e Connessione:**

Un'altra innovativa funzionalità in fase di sviluppo è il sistema di messaggistica in-app. Questo strumento permetterà agli utenti di comunicare direttamente all'interno di AsilApp. Questa caratteristica favorirà la creazione di una rete solidale all'interno della community, dove gli utenti potranno scambiarsi consigli, condividere le proprie storie, e connettersi con persone che condividono gli stessi interessi e le stesse sfide.

- **Integrazione di Sistemi AI per Diagnosi Precoce:**

Introduzione di sistemi basati sull'Intelligenza Artificiale (AI) per l'analisi dei dati medici dei pazienti.
Utilizzo di algoritmi per individuare precocemente eventuali patologie o anomalie nei dati, consentendo interventi tempestivi.

Con queste nuove funzionalità in arrivo, AsilApp si propone di diventare il punto di riferimento per i richiedenti asilo, per i titolari di protezione internazionale e ovviamente per i pazienti in generale, senza lasciare nessuno escluso. L'obiettivo è creare una rete attiva e coinvolgente, dove gli utenti possono trovare tutto ciò di cui hanno bisogno. AsilApp sarà il luogo dove trovare supporto, informazioni utili e connessioni significative per garantire una migliore qualità di vita e un futuro luminoso per TUTTI.

AsilApp